Stakeholder Agent System

Architecture Document

Regis University - MSSE 692: Software Engineering Practicum I

Matt Bass, Anthony Perez, Joseph Turner, Ramesh Kondala
2-1-2026

Executive Summary

This document describes the architecture of the Stakeholder Agent System, an AI-driven web application designed to provide software engineering students with realistic, repeatable practice in requirements elicitation. The system addresses a recurring challenge in academic environments: limited access to real stakeholders for authentic elicitation exercises. The architecture presented here is derived directly from the system's functional and quality attribute requirements defined in Week 2 and reflects deliberate trade-offs appropriate for an 8-week practicum project.

The selected architectural approach is a multi-tier web application architecture. The system is organized into distinct logical tiers: a presentation tier implemented as a browser-based web frontend, an application tier implemented as a backend API, and a data tier providing durable persistence for users, sessions, chat history, and elicited requirements. In addition, the system integrates with external platform services for authentication and AI inference. This structure provides a clear separation of concerns while keeping the overall design simple enough to develop, deploy, and maintain within the project's time and resource constraints.

Several key architectural decisions define the system. First, session durability and recovery are treated as first-class concerns: the backend application tier is designed to be stateless, with all conversational state persisted in the data tier so students can resume work after interruptions or failures. Second, AI integration is isolated behind a controlled adapter layer, allowing the system to handle retries, enforce guardrails, and remain resilient to AI provider outages without impacting user data. Third, authentication is delegated to an external identity provider, reducing security risk and development effort while ensuring strict per-user data isolation.

The architecture primarily addresses three quality attribute priorities. Availability and reliability are achieved through durable storage, stateless services, and graceful degradation during external service failures. Performance is supported through asynchronous AI calls, responsive UI behavior, and efficient prompt construction to meet the requirement of sub-5-second responses for most interactions. Modifiability is enabled through clear tier boundaries, externalized configuration for personas and scenarios, and provider abstraction for AI services.

Several trade-offs are consciously accepted. The backend application tier is implemented as a single logical service rather than decomposed into microservices, prioritizing simplicity and delivery speed over fine-grained scalability. Advanced features such as multi-user sessions, instructor dashboards, and automated grading are deferred to future iterations. These trade-offs are appropriate for an MVP developed under academic constraints and provide a solid architectural foundation for future extension.

Architectural Drivers Analysis

Architectural drivers are the quality attributes and constraints that most strongly influence the system's structure and major design decisions. While the Stakeholder Agent System has many requirements, only a subset materially shapes the architecture. Based on Week 2 quality attribute scenarios and MVP constraints, the following are identified as the top architectural drivers.

Driver :1Availability / Reliability (Session Durability and Recovery)

Referenced Week 2 Scenario: Availability / Reliability Scenario (AI provider downtime, service interruption, resume exact session state)

The system must preserve elicitation history and allow students to resume their sessions after interruptions, including temporary AI provider failures (e.g., 503 errors), application restarts, or transient infrastructure issues. If this driver is not addressed architecturally, the system fails its primary educational purpose: students will lose progress, repeat work, or abandon the tool due to unreliability. This driver directly influences the multi-tier architecture by requiring a durable data tier and a stateless application tier capable of recovering state from persistent storage.

Key architectural implications:

- Durable storage of sessions, chat history, and elicited requirements in the data tier
- Stateless backend application tier that can restart without losing session state
- Controlled AI integration with retries and circuit breaker patterns to degrade gracefully during AI outages
- Clear user-facing error handling that preserves session continuity

Driver 2: Performance (Responsive Conversational Interaction)

Referenced Week 2 Scenario: Performance Scenario (≤ 5 seconds for 95% of responses at ~30 concurrent sessions; UI remains responsive)

The system must provide a responsive conversational experience. If AI responses are slow or the UI becomes unresponsive during AI calls, students will disengage and the elicitation practice will be less effective. This driver shapes both the presentation and application tiers. The frontend must remain usable while awaiting AI responses, and the backend must optimize AI request handling, prompt size, and outbound connection reuse.

Key architectural implications:

- Asynchronous/non-blocking AI call handling in the application tier
- Frontend behavior that does not freeze during AI calls (responsive UI while waiting)
- Token budgeting and prompt construction strategies to reduce LLM latency and cost
- Basic rate limiting and load management to maintain performance under peak use

Driver 3: Modifiability (Evolving Personas, Scenarios, and Providers)

Referenced Week 2 Scenario: Modifiability Scenario (add/update personas and scenarios through configuration; provider flexibility)

Although the MVP begins with a limited scenario and a single stakeholder persona, the system must support future evolution to remain valuable beyond the practicum. The ability to add or update stakeholder personas, project scenarios, and AI provider choices without architectural rework is essential to prevent the system from becoming brittle or “one-off.” This driver strongly influences the multi-tier architecture by emphasizing clear separation of concerns and stable interfaces between internal components and external services.

Key architectural implications:

- Externalized configuration for persona and scenario definitions (data-driven rather than hard-coded)
- An AI provider adapter/facade boundary so that switching providers does not affect session logic
- Modular boundaries within the application tier (session management, guardrails, prompt building, persistence access)
- Tests focused on core behaviors (resume session, isolation, guardrails) to enable safe changes

Architecture Overview

Architectural Style

The Stakeholder Agent System is designed using multi-tier web application architecture. The system separates concerns across distinct logical tiers: presentation, application, and data, while relying on external platform services for authentication and AI inference. This structure supports reliability, performance, and modifiability goals while remaining simple enough to deliver within the practicum's constraints.

Key Structural Elements

- **Front-End Application:**
  A browser-based frontend web application that provides a chat interface, displays session history, and manages user interactions. Programmed in Typescript with TanStack Start
- **AI Service:**
  A backend API responsible for core business logic, including session lifecycle management, AI prompt orchestration, authorization enforcement, and guardrail application. Programmed in Python with Pydantic AI
- **Data Store:**
  Likely AWS ES2. There are many possibilities. Will consider other options as needed.
- External Platform Services:
  - **Authentication Provider**
    for identity and access control
  - **AI Provider**
    accessed via a controlled adapter layer
  - **Cloud Hosting Provider**
    leaning towards 25 Gig free AWS account.

Integration Approach

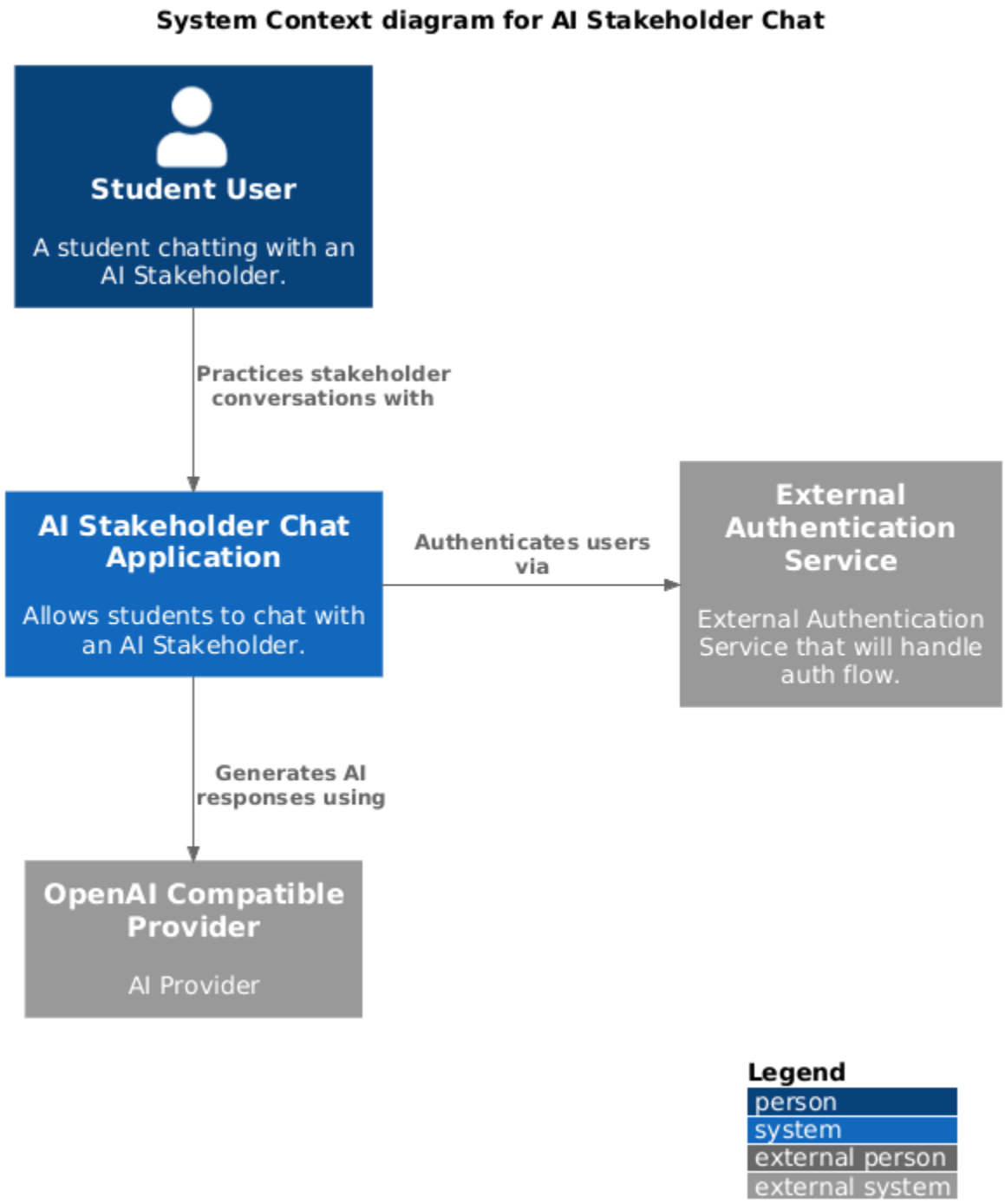
The Front-End Application will make structured calls through the AI Service abstract base class(Interface) to the underlying AI Service. The Auth Integration element in the Application layer will authenticate users through its API once a provider is selected. The AI Service will make SQL calls to the data store to persist or restore session data. The AI Service will call to the backend LLM through its API. The request (question) from the user and the assembled context will be sent to the backend LLM.

Deployment Architecture

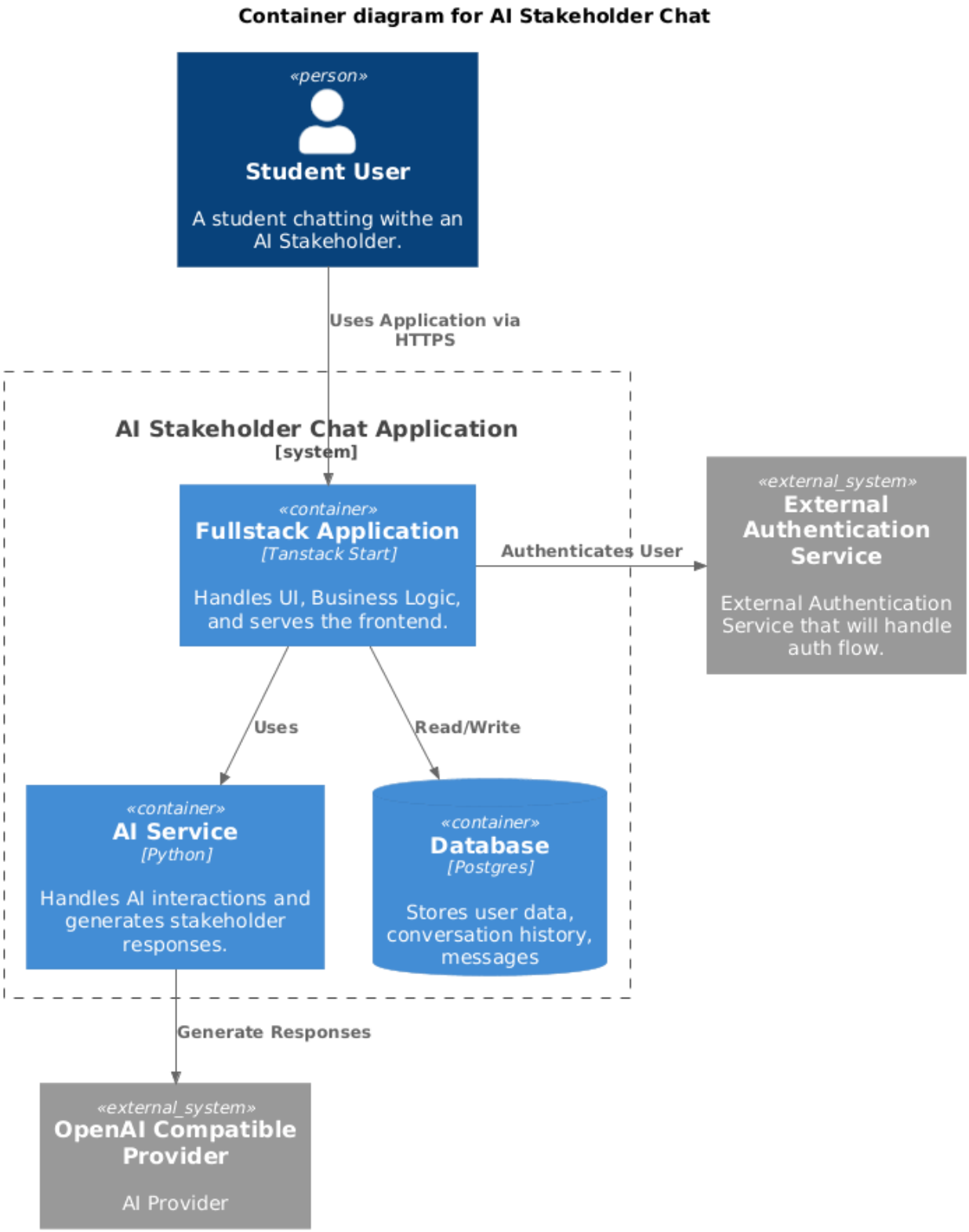
The front-end and backend are deployed as separate components within a cloud environment. The backend is stateless (build context for each request), allowing for restart or redeployment without loss of user data. The database provides durable storage, while authentication and AI services are consumed as managed external dependencies.

Architecture Views (C4 Model)

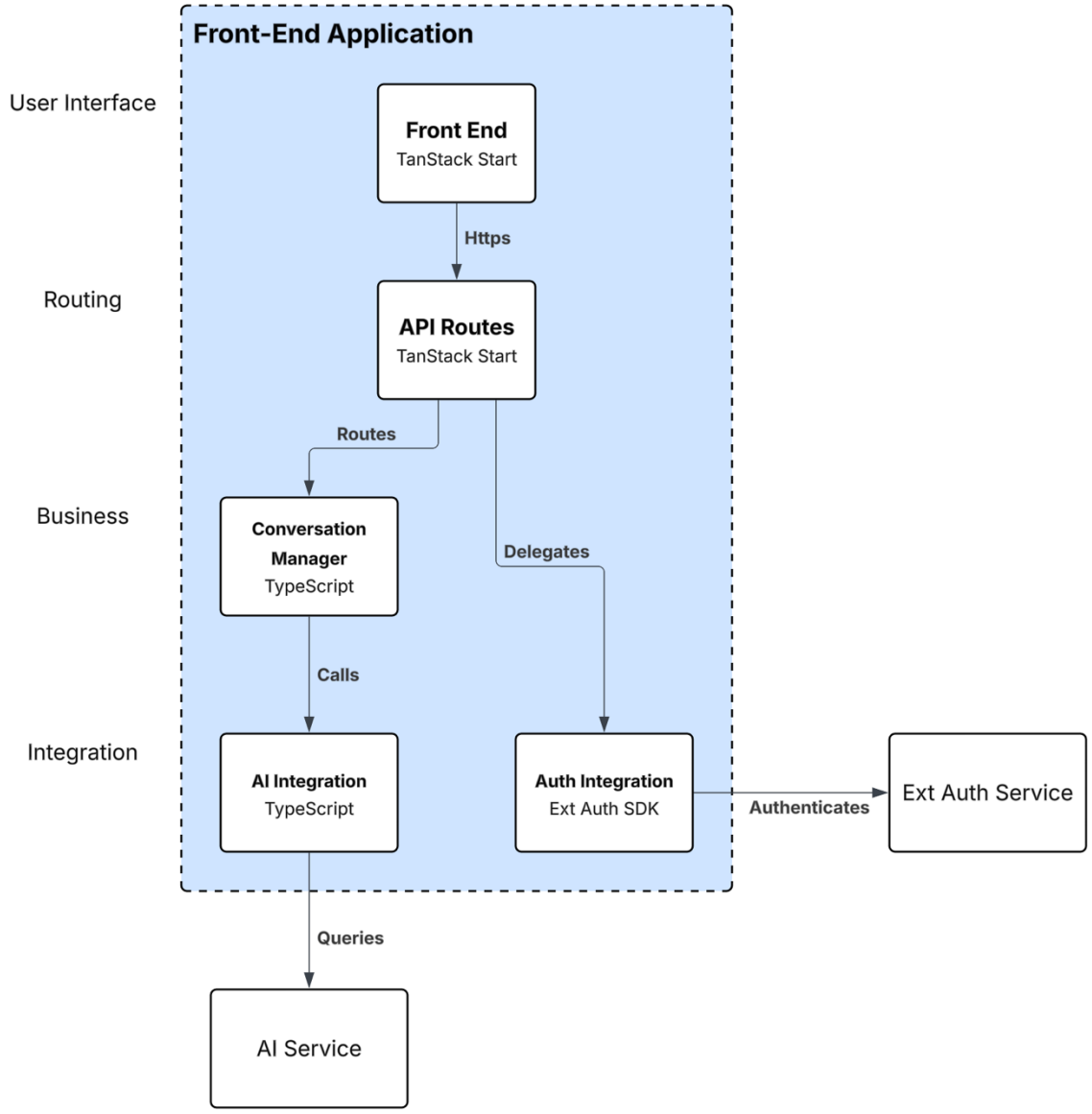
Level 1: Context Diagram



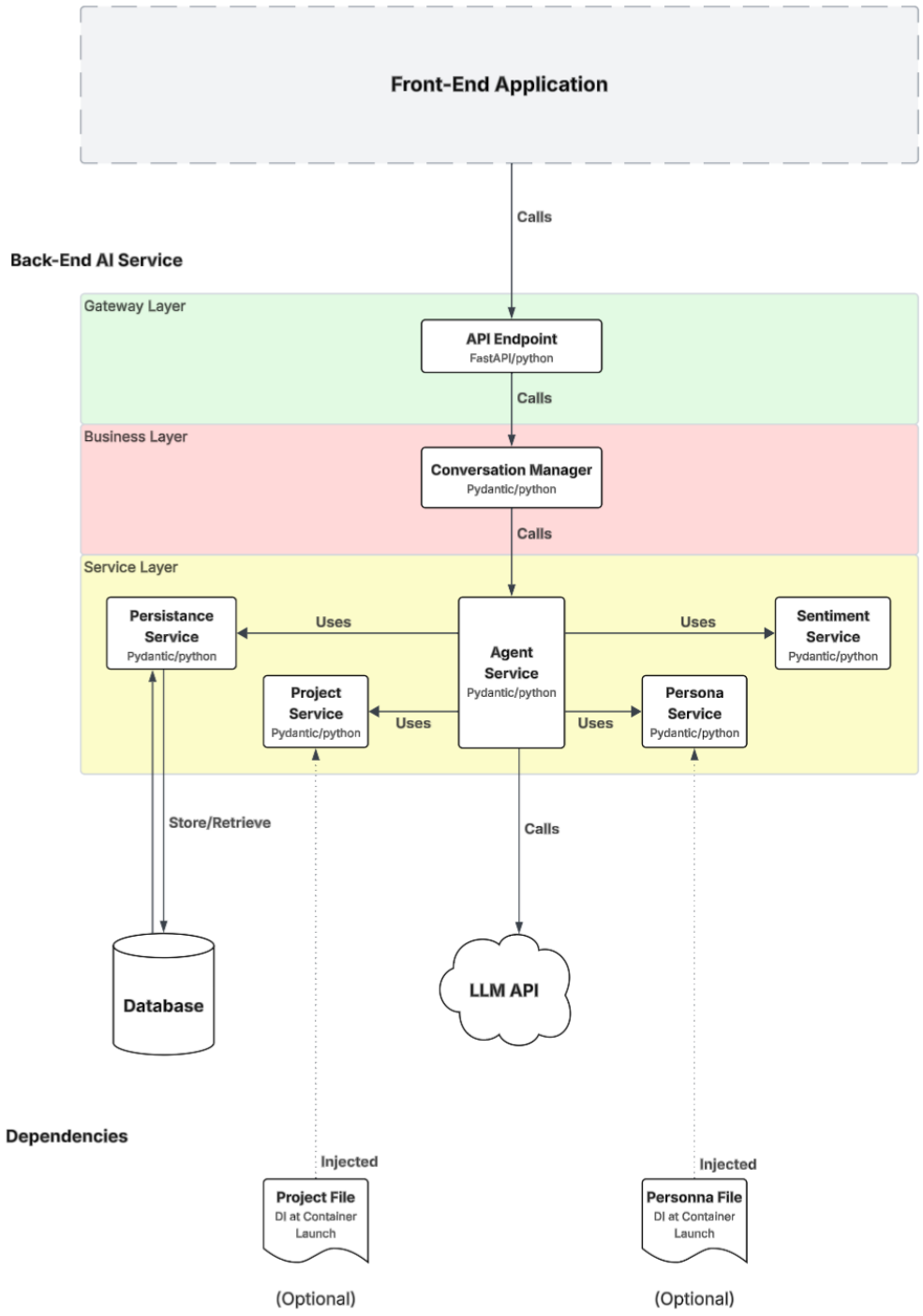
Level 2: Container Diagram



Level 3: Component Diagram 1 (Application)



Level 3: Component Diagram 2 (AI Service)



Quality Attribute Achievement Strategy

QA Scenario: Performance

Tactics Applied:

- Handled AI calls asynchronously using non-blocking I/O.
- Storing the stable context (such as scenario, persona, and key facts) helps to keep the prompt size smaller and more manageable.
- Utilized connection pooling to optimize AI and client libraries.
- Basic load and rate limiting mechanisms to prevent overload.

How Achieved:

- The backend efficiently handles AI requests by sending them asynchronously. It streams or returns responses without blocking the UI thread, ensuring the browser stays responsive and smooth while waiting.
- Static or slowly changing context, like project descriptions or persona profiles, is stored separately and reused in prompts. This approach helps reduce latency and token costs by avoiding the need to resend the entire conversation each time.
- The AI client is set up with connection pooling, allowing it to be reused across requests to help reduce the overhead of setting up new connections.
- Simple rate limiting is implemented per user or session to prevent accidental overloads, such as spamming the “send” button. This helps keep the P95 latency at or below 5 seconds during the expected peak load.

QA Scenario: Availability / Reliability

Tactics Applied:

- Retried using exponential backoff to handle temporary AI/API failures.
- Implemented a circuit breaker for external AI provider access.
- Reliable storage of session and conversation state.
- Health checks alongside basic monitoring and alerts.

How Achieved:

- For temporary issues like 5xx errors from the AI provider, the system kindly retries a set number of times with a gentle backoff before showing an error. This helps reduce the chances of students seeing failures.
- A circuit breaker tracks ongoing AI failures; when activated, it halts provider calls briefly and shows a friendly “temporarily unavailable” message while maintaining state, avoiding cascading timeouts.
- Conversation history and requirements are stored securely in a trusted database, allowing students to pick up right where they left off even if the app restarts. This way, they won’t lose their progress and can continue smoothly.

- We carefully monitor application and dependency health endpoints. When we notice repeated failures or high error rates, we promptly send alerts to our team so they can step in early and help prevent a significant impact on students.

QA Scenario: Security

Tactics Applied:

- Authentication and role-based access control (RBAC).
- Data isolation at the tenant or user level and queries are limited to only necessary privileges.
- Transport and at-rest encryption.
- Input validation and output filtering / AI guardrails.

How Achieved:

- Access to the app always requires login (such as username/password or OAuth), with each request associated with an authenticated identity. In the future, role distinctions like instructor versus student could limit access to admin and reporting features.
- All database queries are limited by user or session IDs, ensuring students can only view and edit their own sessions. Shared tables are not accessible without specific filters and read/write permissions are kept to a minimum in the backend.
- TLS is used for all network traffic, and the database or managed storage is set up to encrypt data at rest. This helps keep chat logs and other information safe from interception or theft.
- Every user input to the AI is carefully checked and cleaned to make sure everything stays safe and on track. Prompts and responses go through special guardrail checks that prevent off-topic, prompt-injection, or unsafe content, and any suspicious activity is logged for review, helping keep our interactions secure and friendly.

QA Scenario: Modifiability

Tactics Applied:

- We use externalized configurations for scenarios and personas, making things data-driven instead of hard-coded. This approach helps keep things flexible and easy to update.
- This layered architecture thoughtfully separates concerns like UI, domain, infrastructure, and AI adapter, making the system more organized and easier to manage.
- Implementation of abstraction for the AI provider via a pluggable provider interface.
- Automated tests focus on fundamental behaviors.

How Achieved:

- Project scenarios and stakeholder personas are stored as configurations, like JSON or database records. They are loaded at runtime, so whenever you want to add or adjust them, there's no need to change the code, just update the data.
- The system is thoughtfully organized into layers: the front-end takes care of the presentation, the domain layer manages sessions and requirements, and infrastructure adapters handle

persistence and AI calls. This setup makes it easier to update or change one part without affecting the others, offering a flexible and resilient structure.

- AI access is designed through an adaptable interface, allowing providers like OpenAI, Claude, and others to be easily swapped or configured. This way, you can make changes without needing to rewrite your core business logic, making upgrades or adjustments smoother and more straightforward.
- Unit and integration tests thoroughly cover important workflows like resume session, guardrails, and performance constraints. This way, whenever there are changes in scenarios or infrastructure, any regressions are identified quickly, ensuring a smooth and safe development process over many semesters.

Technology Selection Rationale

We have chosen to separate the system into a TypeScript full-stack application and an independent Python AI service. This division directly supports our extensibility requirement for adding new stakeholder personas and project scenarios. Any changes to AI behavior stay isolated in the Python service without risking the core application. This separation also enables our 99.9% uptime goal since we can update, restart, and scale the AI service independently. When the AI service experiences issues, the main application will remain available for students to view their conversation history or manage their profiles.

Given this architecture, Python became the clear choice for the AI service. Python currently dominates the AI ecosystem with libraries like PydanticAI and LangChain that provide the capabilities we need. Beyond the ecosystem support, Python's AI tooling gives us fine-grained control over error handling and response streaming, which is essential for our UI responsiveness requirements around clear error messages for erratic AI behavior. When the AI produces unexpected responses or hits rate limits, we can provide students with specific, actionable feedback rather than generic failures.

The data these services manage follows clear patterns that led us to a relational database. Users own conversations, conversations contain messages, and these relationships are stable and well-defined. A relational database enforces these relationships and directly supports our data resiliency and session continuity requirements. If a student's session is interrupted, we can quickly retrieve the exact conversation state from related tables and resume where they left off. Database transactions ensure conversation history remains consistent even during failures, and the structured schema prevents data corruption that could lose student progress.

For communication between our TypeScript application and Python AI service, we chose REST because student interactions follow a simple request-response pattern: students send a message and receive an AI response. Our 5-second response time goal for AI interactions at 30 concurrent students with 500-character requests is easily achievable with REST. The stateless nature of REST directly supports our 30-second downtime recovery goal. When services restart, clients simply retry their requests without complex state reconstruction. Our retry strategy with exponential backoff maps naturally to REST's request-response model, and standard HTTP status codes make it straightforward to determine which requests should be retried.

Finally, because user data privacy is non-negotiable with student information, we are utilizing an external authentication provider. This provider handles encrypted storage and transmission as their core competency, eliminating our risk of mishandling passwords, sessions, or personal data. They maintain the high availability we need since authentication downtime blocks all student access. Critically, these providers prevent us from accidentally logging sensitive user data during authentication flows, which directly satisfies our requirement that no user data appears in logs or error messages. Building this ourselves would mean ongoing security maintenance that diverts effort from our core development while introducing significant risk.

Architectural Constraints and Assumptions

Constraints

- 8-week development window.
- Limited team architecture experience.
- Distributed team.
- Existing team knowledgebase.
- Extremely limited to no budget.
- Current capabilities and requirements of LLM APIs.
- Current capabilities of AI frameworks and libraries.
- Cost of cloud hosting.
- Requirements for 3rd party integrations (auth, data storage, cloud)
- Requirement for data privacy.

Assumptions

- Continued availability of LLM APIs at no or manageable cost.
- Continued affordable access to cloud hosting.
- Continued availability of 3rd party authentication service.
- Continued availability of 3rd party data storage.
- Currently available AI frameworks and libraries will suit system requirements.
- Required MVP functionality can be implemented in 3 weeks.

Risks and Mitigations

Risk	**Mitigation**
LLM API could become unavailable (company goes bankrupt, etc.).	There are multiple LLM APIs available that could suit requirements.
Cloud providers could go out of business or raise price out of means to extend investment.	design for portability to other cloud provider.
3rd party auth service could become unavailable (provider out of business, raise cost).	Design for modularity of 3rd party integrations.
Cloud provider downtime.	Select a provider with good redundancy mechanisms.
Auth provider downtime.	Select provider with good up-time track record.
User data could be intercepted by unauthorized party when communicating with LLM API.	Encrypt transmitted data.
Saved data could be accessed by unauthorized party.	Encrypt stored data, requires authentication to access.
A user's data could be accessed by an unauthorized individual.	Require login with username/password.
User may want/need to stop/resume elicitation work without losing progress.	Implement save/restore of chat functionality.

Team Architecture Review Summary

Date: 01/30/26 6pm

Attendees: Anthony Perez, Joe Turner, Matt Bass, Ramesh Kondala

The team reviewed and considered 2 competing architectures. Anthony produced an architecture that centered on a TanStack application that calls a python/pydantic lightweight AI service that in turn calls an LLM API on the back end. This architecture located all the AI context logic in the application leaving the AI service as a relatively simple adapter. Matt proposed a unified application that leveraged only python/pydantic. At first the team leaned towards Anthony's concept due to performance concerns. Through discussion it became apparent that locating the context logic in the pydantic service offered numerous benefits to managing the context, so the Two competing architectures were combined.

The result is a TanStack front-end application written in TypeScript and the backend AI service that will contain the context logic to leverage the convenience and efficiency of the pydantic framework. The biggest outstanding concern is the performance of retrieving the conversation history for each backend LLM request. A few different caching solutions were very briefly discussed simply to prove their practicality should they turn out to be needed, but the Minimum Viable Product (mvp) delivered in this short development window likely will not include any performance optimizations.